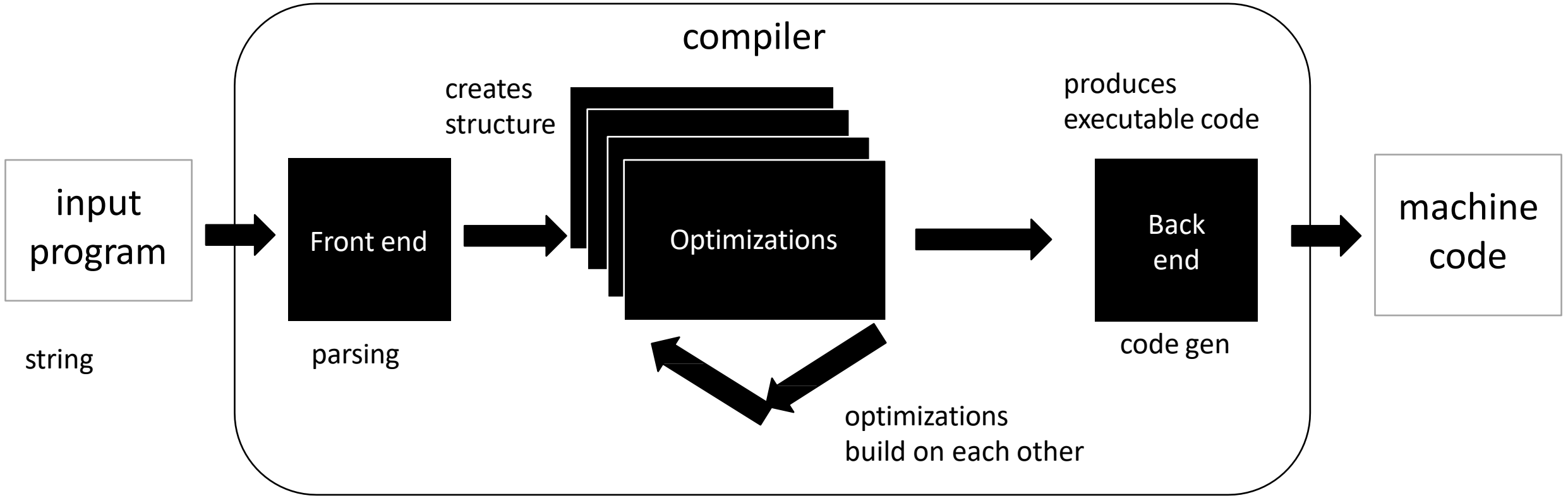
CSE110A: Compilers

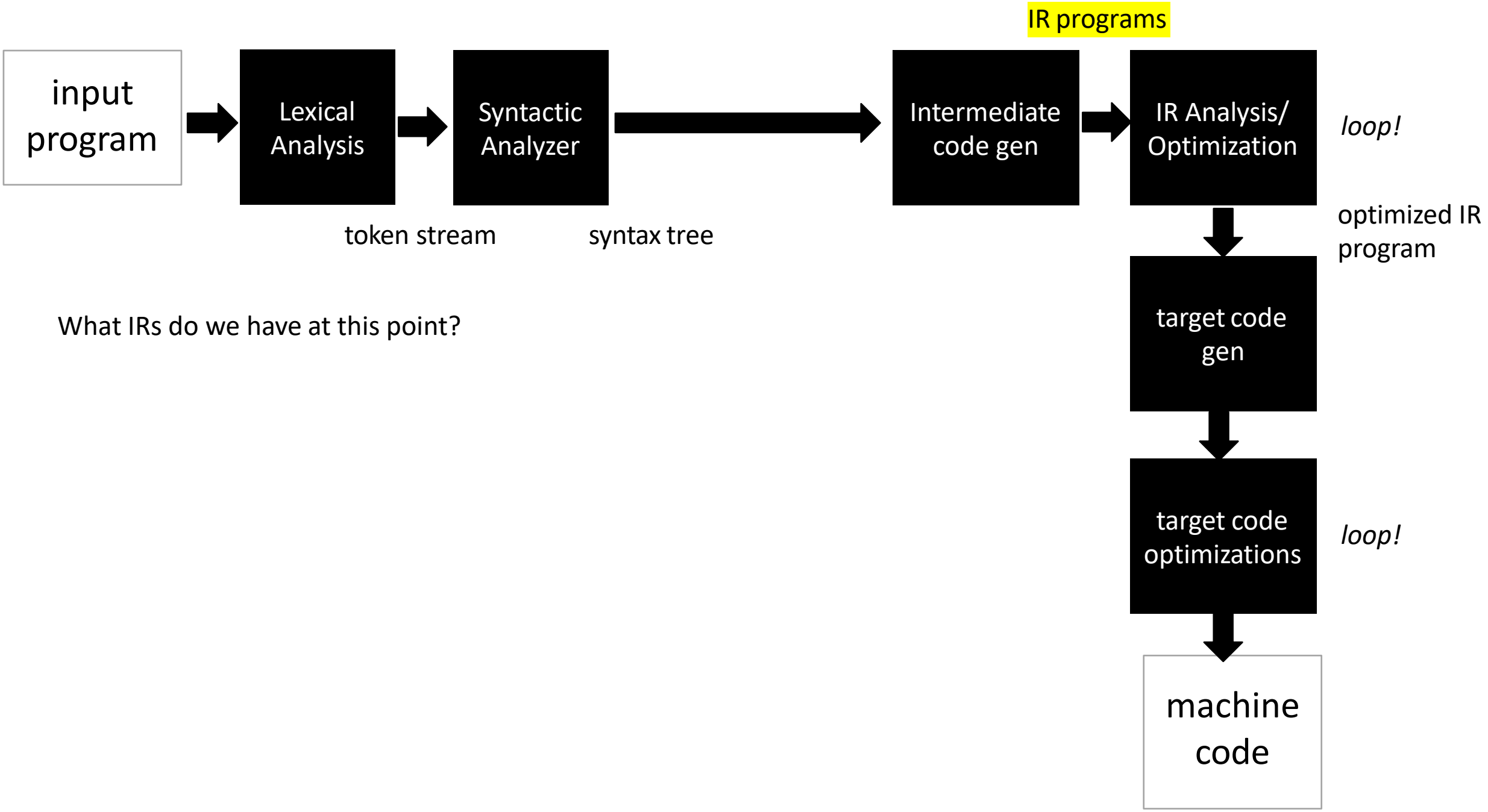
Topics:

- *Start of Module 4 – Optimizations*
  - *Overview*
  - *Basic Blocks (13)*
  - *Local Value Numbering (28)*
  - *Stitching Optimized Code Back Together (62)*

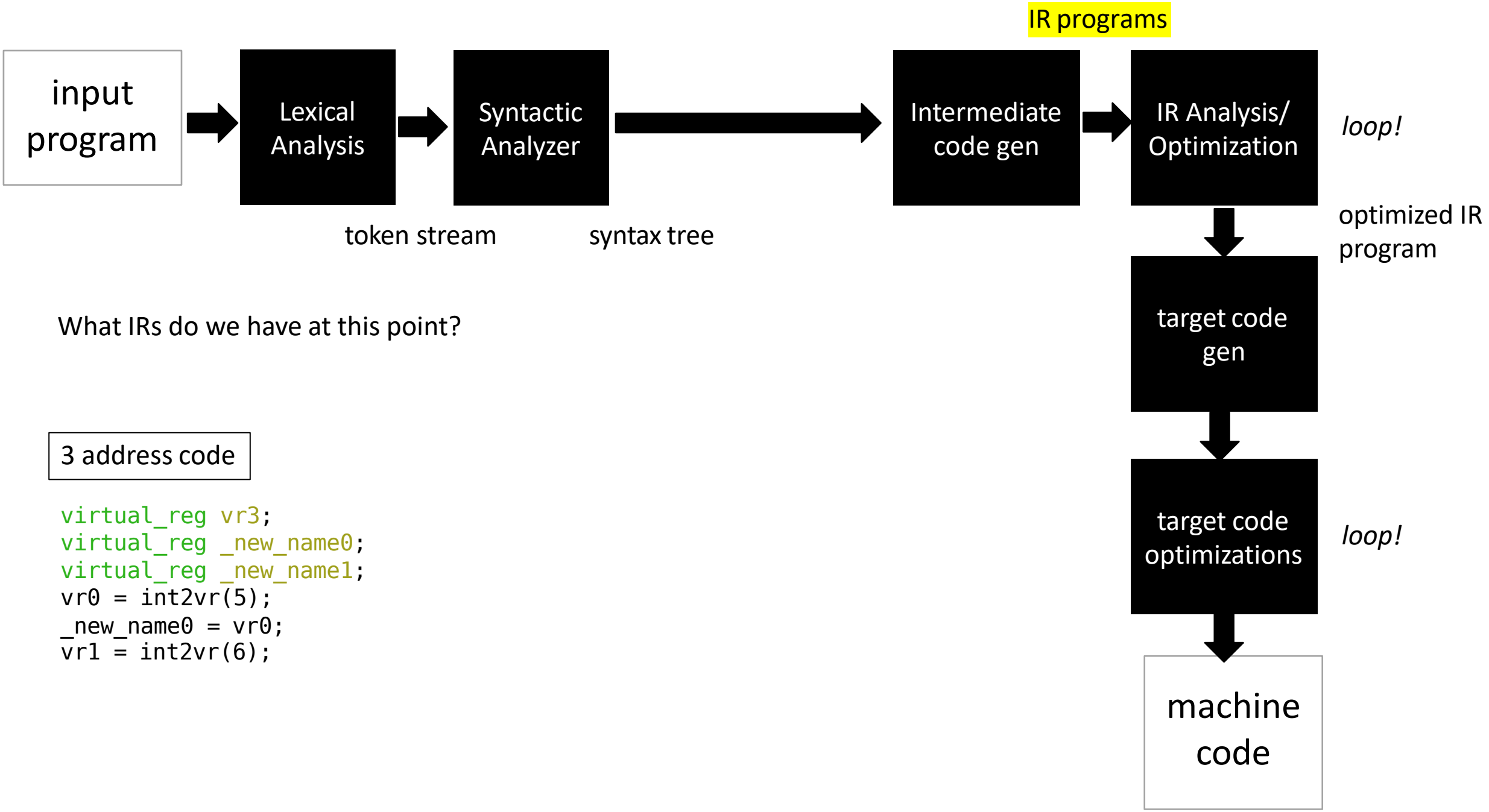
Zooming out again: Compiler Architecture



IRs and type inference type inference are at the boundary of parsing and optimizations



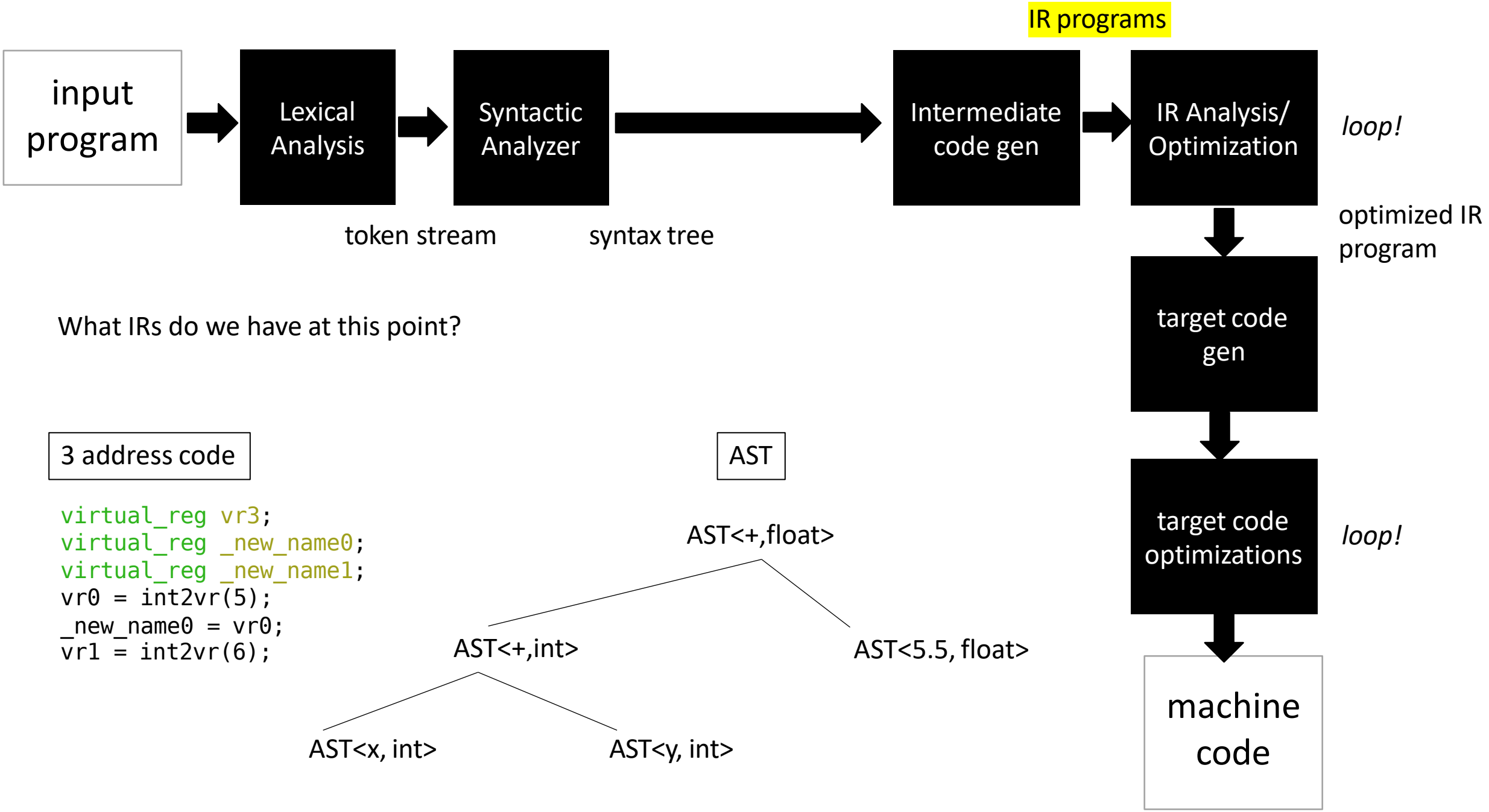
What IRs do we have at this point?



What IRs do we have at this point?

3 address code

```
virtual_reg vr3;
virtual_reg _new_name0;
virtual_reg _new_name1;
vr0 = int2vr(5);
_new_name0 = vr0;
vr1 = int2vr(6);
```



What IRs do we have at this point?

3 address code

```
virtual_reg vr3;
virtual_reg _new_name0;
virtual_reg _new_name1;
vr0 = int2vr(5);
_new_name0 = vr0;
vr1 = int2vr(6);
```

AST

Implicit parse tree

```
if_else_statement := IF LPAR expr RPAR statement ELSE statement
```

```
if (program0) {
  program1
}
else {
  program2
}
```

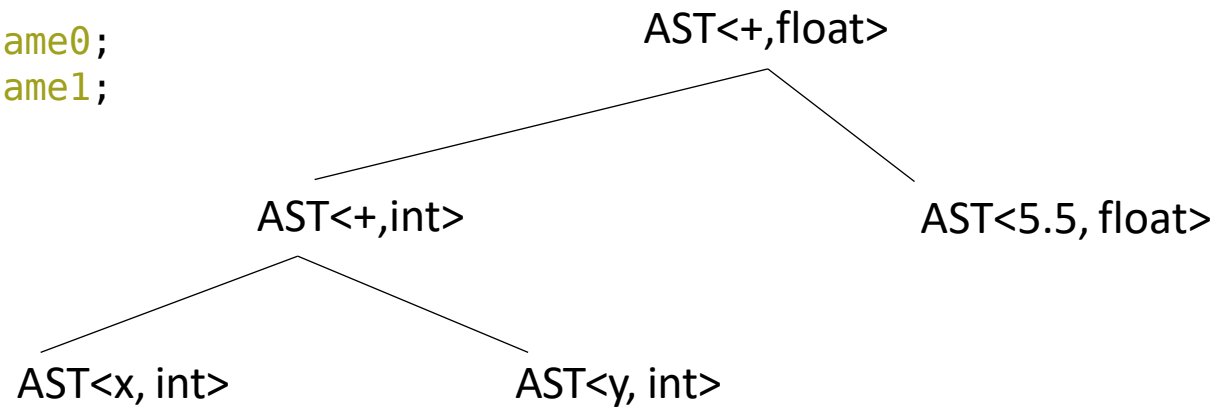
We have several structures to utilize to analyze and optimize programs!

What IRs do we have at this point?

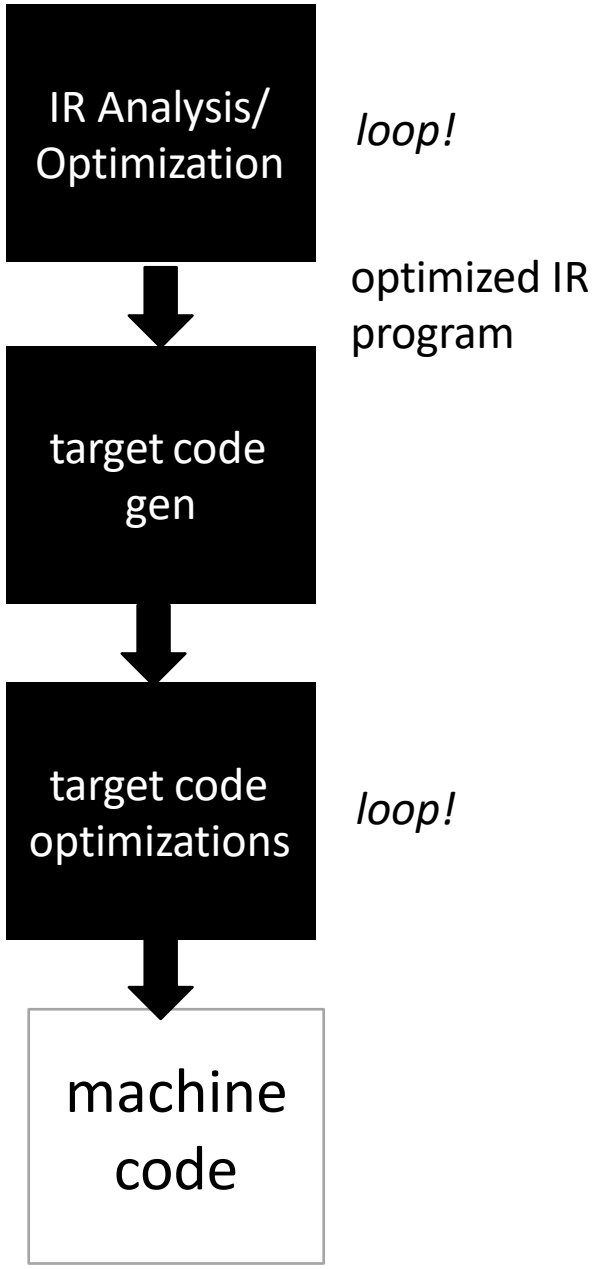
3 address code

```
virtual_reg vr3;
virtual_reg _new_name0;
virtual_reg _new_name1;
vr0 = int2vr(5);
_new_name0 = vr0;
vr1 = int2vr(6);
```

AST



IR programs



Optimization categories

- Machine-independent - these optimizations should work well across many different systems
  - Examples?

- Machine dependent - these optimizations start to optimize the code for a given system
  - Examples?

Optimization categories

- Machine-independent - these optimizations should work well across many different systems
  - Examples?
  - All the examples we looked at before seem like they will help across many systems
- Machine dependent - these optimizations start to optimize the code for a given system
  - Examples?
  - loop chunking for cache line size and vectorization.
  - instruction re-orderings to take advantage of processor pipelines.
  - fused multiply-and-add instructions
  - vector processing (parallel computation)

Optimization categories

- **Machine-independent** - these optimizations should work well across many different systems
  - Examples?
  - All the examples we looked at before seem like they will help across many systems
- In this module we will be looking at machine-independent optimizations.
- What are the pros of machine-independent optimizations?

Optimization categories

Next category level is how much code we need to reason about for the optimization.

- **Local optimizations**: examine a ”basic block”, i.e. a small region of code with no control flow.
  - Examples?
- **Regional optimizations**: several basic blocks with simple control flow.
  - Examples?
- **Global optimization:** optimizes across an entire function

Optimization categories

- **Local optimizations**: examine a ”basic block”, i.e. a small region of code with no control flow.
- **Regional optimizations**: several basic blocks with simple control flow
- **Global optimization:** optimizes across an entire function
- **IDFA Inter-Procedural Data Flow Analysis** : Optimizes across functions

Discussion:

- What are the pros and cons of each?
- Going across functions is less common, why?

Optimization categories

- **Local optimizations**: examine a ”basic block”, i.e. a small region of code with no control flow.
- **Regional optimizations**: several basic blocks with simple control flow
- **Global optimization:** optimizes across an entire function

For this module:

- We will look at two optimizations in detail:
- A local optimization: Local value numbering
- A regional optimization: Loop unrolling
- We will implement both as homework
- We will discuss several other optimizations and analysis

BASIC BLOCKS

IR Program structure

- A sequence of 3 address instructions

- Programs can be split into **Basic Blocks:**
  - A **sequence of 3 address instructions** such that:
    there is **a single entry, single exit**

- *Important property*: an instruction in a basic block can assume that all preceding instructions will execute

Single Basic Block

```
Label_x:
op1;
op2;
op3;
br label_z;
```

IR Program structure

- A sequence of 3 address instructions
- Programs can be split into **Basic Blocks:**
  - A **sequence of 3 address instructions** such that: there is **a single entry, single exit**
- *Important property*: an instruction in a basic block can assume that all preceding instructions will execute

Single Basic Block

```
Label_x:
op1;
op2;
op3;
br label_z;
```

Two Basic Blocks

```
Label_x:
op1;
op2;
op3;

Label_y:
op4;
op5;
```

IR Program structure

How might they appear in a high-level language? What are some examples?

- A sequence of 3 address instructions
- Programs can be split into **Basic Blocks:**
  - A **sequence of 3 address instructions** such that: there is **a single entry, single exit**
- *Important property*: an instruction in a basic block can assume that all preceding instructions will execute

Single Basic Block

```
Label_x:
op1;
op2;
op3;
br label_z;
```

Two Basic Blocks

```
Label_x:
op1;
op2;
op3;

Label_y:
op4;
op5;
```

IR Program structure

- A sequence of 3 address instructions
- Programs can be split into **Basic Blocks:**
  - A **sequence of 3 address instructions** such that: there is **a single entry, single exit**
- *Important property*: an instruction in a basic block can assume that all preceding instructions will execute

How might they appear in a high-level language?

How many basic blocks?

```
…
if (x) {
    …
}
else {
   …
}
…
```

Two Basic Blocks

```
Label_x:
op1;
op2;
op3;

Label_y:
op4;
op5;
```

Single Basic Block

```
Label_x:
op1;
op2;
op3;
br label_z;
```

Converting 3 address code into basic blocks

- Let’s try an example: test 4 in HW 4:

Converting 3 address code into basic blocks

- Simple algorithm:
  - keep a list of basic blocks
  - a basic block is a list of instructions

  - Iterate over the 3 address instructions
  - if you see a branch or a label, finalize the current basic block and start a new one.

Converting 3 address code into basic blocks

```
basic_blocks = []
bb = []

for instr in program:
    # If label starts a new block
    if instr.type == "label":
        if bb:                  # close previous block
            basic_blocks.append(bb)
        bb = [instr]            # start new block
    else:
        bb.append(instr)
        # If branch ends a block
        if instr.type == "branch":
            basic_blocks.append(bb)
            bb = []

# append final block if non-empty
if bb:
    basic_blocks.append(bb)
```

Optimization levels

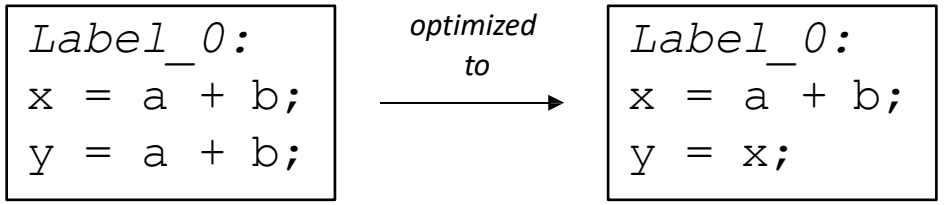
- **Local optimizations:**
  - Optimizes an individual basic block
- **Regional optimizations:**
  - Combines several basic blocks
- **Global optimizations:**
  - operates across an entire procedure
  - what about across procedures?

Optimization levels

```
Label_0:
x = a + b;
y = a + b;
```

- **Local optimizations:**
  - Optimizes an individual basic block
- **Regional optimizations:**
  - Combines several basic blocks
- **Global optimizations:**
  - operates across an entire procedure
  - what about across procedures?

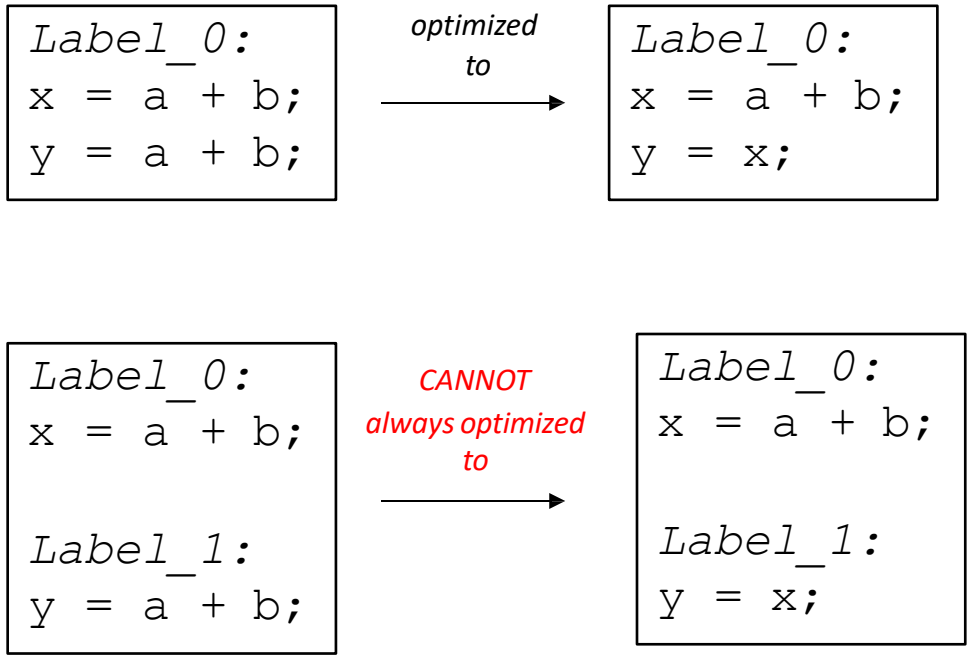
Optimization levels



- **Local optimizations:**
  - Optimizes an individual basic block
- **Regional optimizations:**
  - Combines several basic blocks
- **Global optimizations:**
  - operates across an entire procedure
  - what about across procedures?

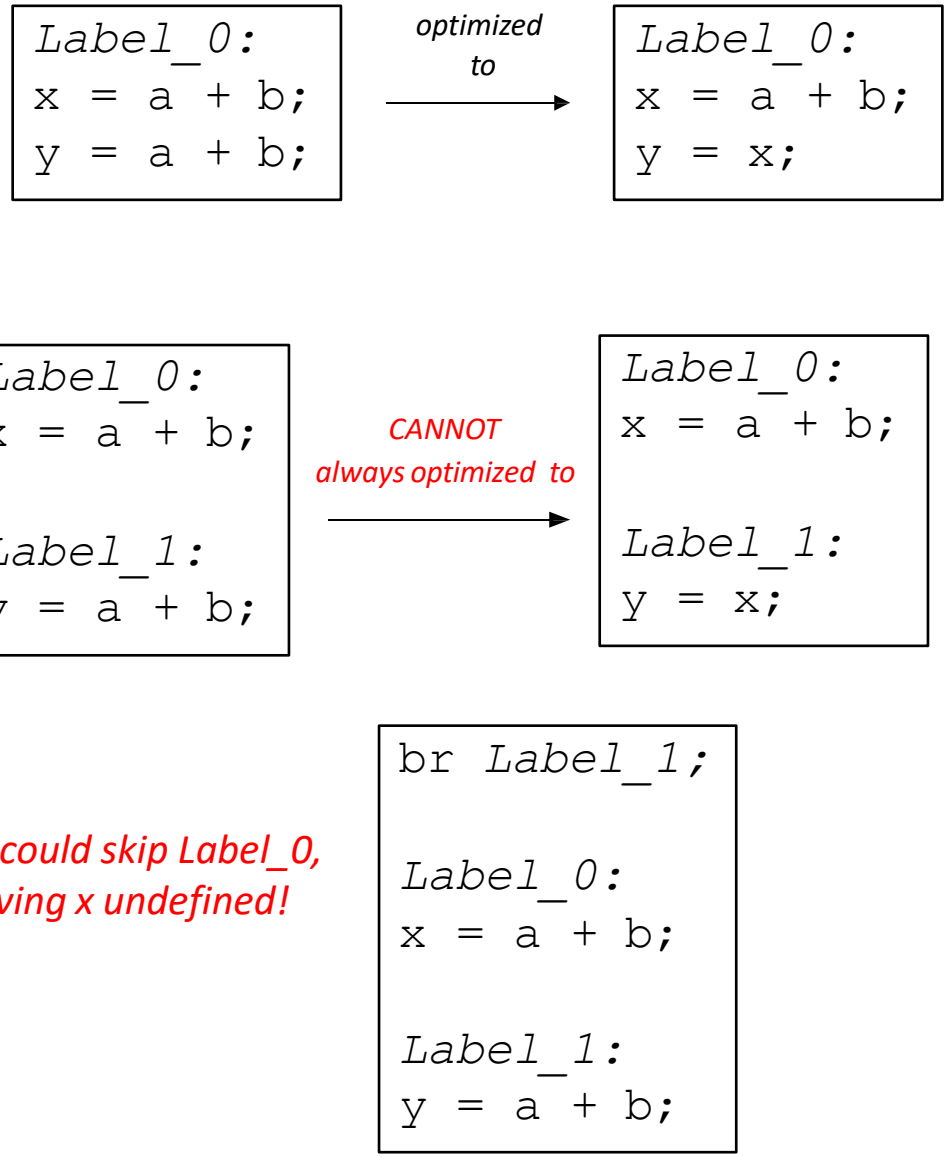
Optimization levels

- **Local optimizations:**
  - Optimizes an individual basic block
- **Regional optimizations:**
  - Combines several basic blocks
- **Global optimizations:**
  - operates across an entire procedure
  - what about across procedures?



Optimization levels

- **Local optimizations:**
  - Optimizes an individual basic block
- **Regional optimizations:**
  - Combines several basic blocks
- **Global optimizations:**
  - operates across an entire procedure
  - what about across procedures?



Regional Optimization

```
…
if (x) {
    …
}
else {
  x = a + b;
}
y = a + b;
…
```

we cannot replace:
y = a + b.
with
y = x;

Regional Optimization: Data Flow Analysis

```
…
if (x) {
    …
}
else {
  x = a + b;
}
y = a + b;
…
```

we cannot replace:
y = a + b.
with
y = x;

```
x = a + b;
if (x) {
    …
}
else {
  …
}
y = a + b;
…
```

Can x = a + b be hoisted out of the if then else?

In that case, we can check if a and b are not redefined, then
y = a + b;
can be replaced with
y = x;

This requires regional analysis and optimizations

Local value numbering

- A local optimization over 3 address code
- Attempts to replace arithmetic operations (expensive) with copy instructions (cheap)
- Can be extended to a regional optimization using flow analysis

Local value numbering

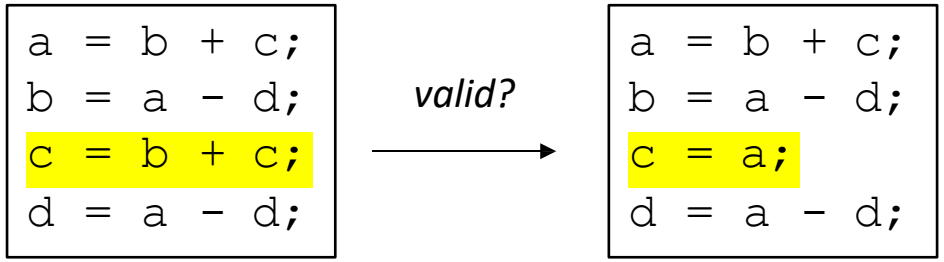
- A local optimization over 3 address code
- Attempts to replace arithmetic operations (expensive) with copy instructions (cheap)
- Can be extended to a regional optimization using flow analysis

```
a = b + c;
b = a - d;
c = b + c;
d = a - d;
```

We are going to use notation similar to Class IR applicable to any Intermediate representation.

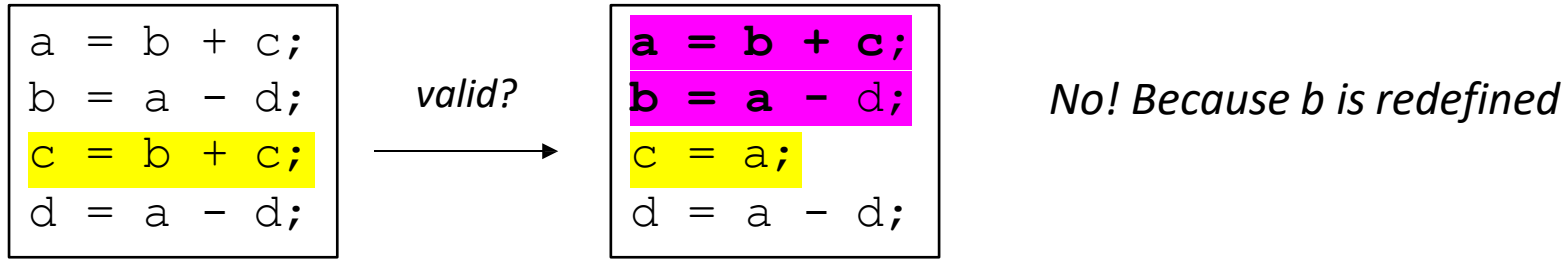
Local value numbering

- A local optimization over 3 address code
- Attempts to replace arithmetic operations (expensive) with copy instructions (cheap)
- Can be extended to a regional optimization using flow analysis



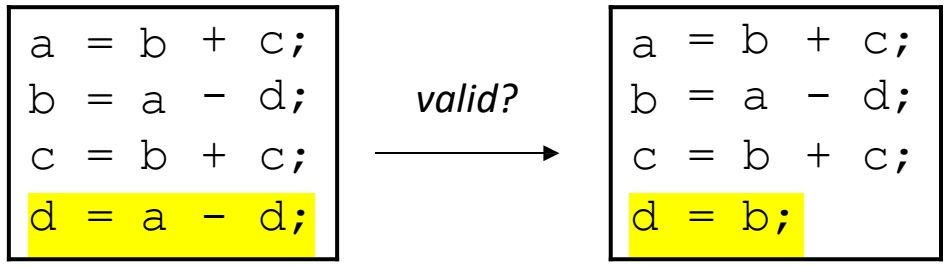
Local value numbering

- A local optimization over 3 address code
- Attempts to replace arithmetic operations (expensive) with copy instructions (cheap)
- Can be extended to a regional optimization using flow analysis



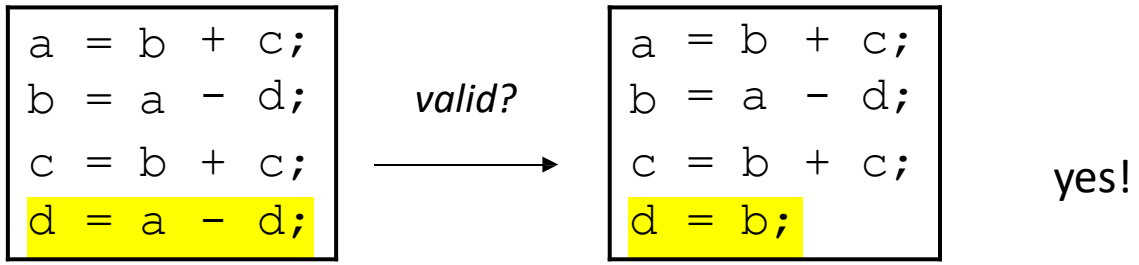
Local value numbering

- A local optimization over 3 address code
- Attempts to replace arithmetic operations (expensive) with copy instructions (cheap)
- Can be extended to a regional optimization using flow analysis



Local value numbering

- A local optimization over 3 address code
- Attempts to replace arithmetic operations (expensive) with copy instructions (cheap)
- Can be extended to a regional optimization using flow analysis



yes!

Local value numbering (LVN)

LVN Algorithm:

- Provide a number to each variable. Update the number each time the variable is updated.

- Keep a global counter; increment with new variables or assignments

```
a = b + c;
b = a - d;
c = b + c;
d = a - d;
```

Global_counter = 0

Local value numbering

Algorithm:

- Provide a number to each variable. Update the number each time the variable is updated.

- Keep a global counter; increment with new variables or assignments

```
a = b0 + c1;
b =  a - d;
c =  b + c;
d =  a - d;
```

Global_counter = 2

b and c have not been seen, give them a number

Local value numbering

Algorithm:

- Provide a number to each variable. Update the number each time the variable is updated.

- Keep a global counter; increment with new variables or assignments

```
a2 = b0 + c1;
b =  a2 - d3;
c =  b  + c;
d =  a  - d;
```

Global_counter = 4

a2 have been updated use the last defined number. i.e. a on RHS, becomes a2 and so on.

Local value numbering

Algorithm:

- Provide a number to each variable. Update the number each time the variable is updated (assigned to).

- Keep a global counter; increment with new variables or assignments

```
a2 = b0 + c1;
b4 = a2 - d3;
c5 = b4 + c1;
d6 = a2 - d3;
```

Global_counter = 7

So … for LHS variable always increment with updated number, for right hand side use, always appended last number version

Local value numbering

Algorithm: Now that variables are numbered

- Iterate sequentially through instructions. **Keep a hash table of the rhs (numbered variables and operation) mapped to their lhs**.

- At each step, check to see if the rhs has already been computed.

```
a2 = b0 + c1;
b4 = a2 - d3;
c5 = b4 + c1;
d6 = a2 - d3;
```

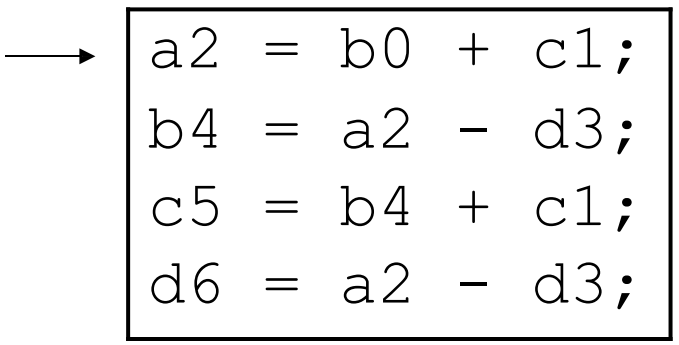
If we remember to declare every one of these variable names, the IR is still valid !!!

Local value numbering

Algorithm: Now that variables are numbered

- Iterate sequentially through instructions. **Keep a hash table of the rhs (numbered variables and operation) mapped to their lhs**.

- At each step, check to see if the rhs has already been computed.



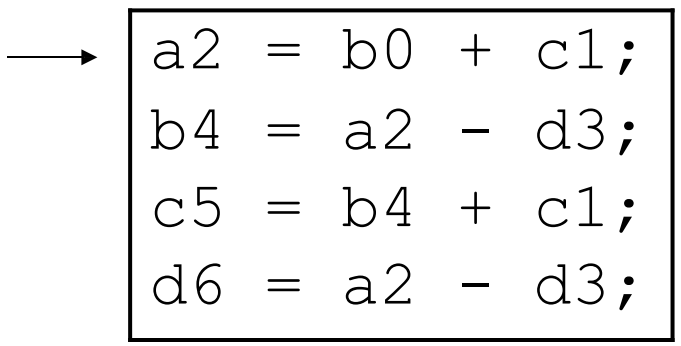

```
H = {
}
```

Local value numbering

Algorithm: Now that variables are numbered

- Iterate sequentially through instructions. **Keep a hash table of the rhs (numbered variables and operation) mapped to their lhs.**

- At each step, check to see if the rhs has already been computed.



```
a2 = b0 + c1;
b4 = a2 - d3;
c5 = b4 + c1;
d6 = a2 - d3;
```

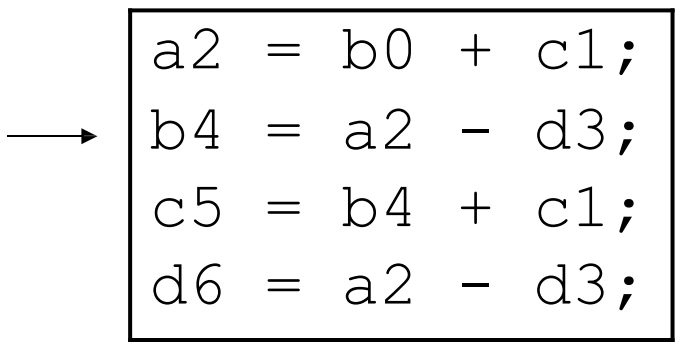
```
H = {
        "b0 + c1" : "a2",
}
```

Local value numbering

Algorithm: Now that variables are numbered

- Iterate sequentially through instructions. Keep a hash table of the rhs (numbered variables and operation) mapped to their lhs.

- At each step, check to see if the rhs has already been computed.



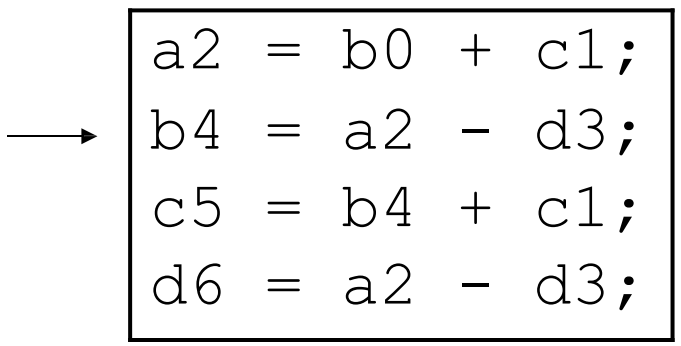

```
H = {
        "b0 + c1" : "a2",
}
```

Local value numbering

Algorithm: Now that variables are numbered

- Iterate sequentially through instructions. **Keep a hash table of the rhs (numbered variables and operation) mapped to their lhs.**

- At each step, check to see if the rhs has already been computed.



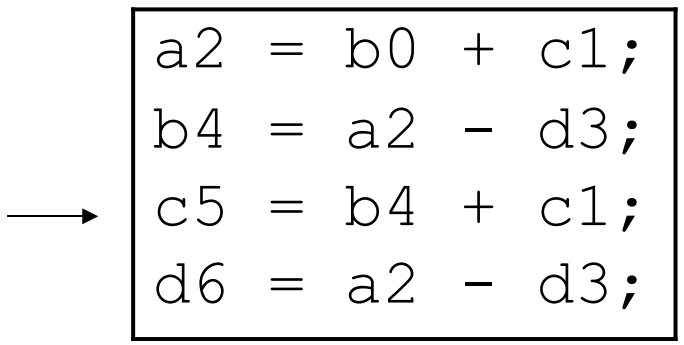

```
H = {
        "b0 + c1" : "a2",
        "a2 - d3" : "b4",
}
```

Local value numbering

Algorithm: Now that variables are numbered

- Iterate sequentially through instructions. Keep a hash table of the rhs (numbered variables and operation) mapped to their lhs.

- At each step, check to see if the rhs has already been computed.



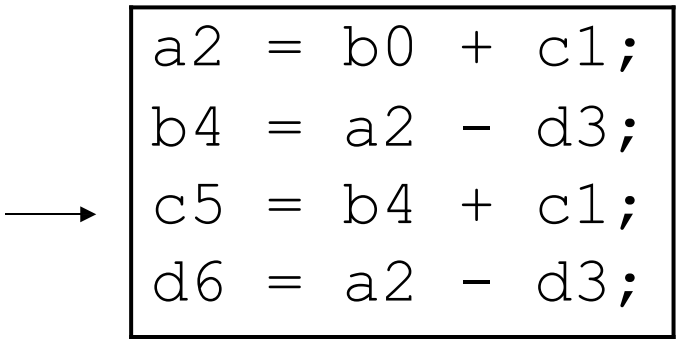

```
H = {
        "b0 + c1" : "a2",
        "a2 - d3" : "b4",
}
```

Local value numbering

Algorithm: Now that variables are numbered

- Iterate sequentially through instructions. Keep a hash table of the rhs (numbered variables and operation) mapped to their lhs.

- At each step, check to see if the rhs has already been computed.




```
H = {
        "b0 + c1" : "a2",
        "a2 - d3" : "b4",
}
```

mismatch due to re-numbering!!!

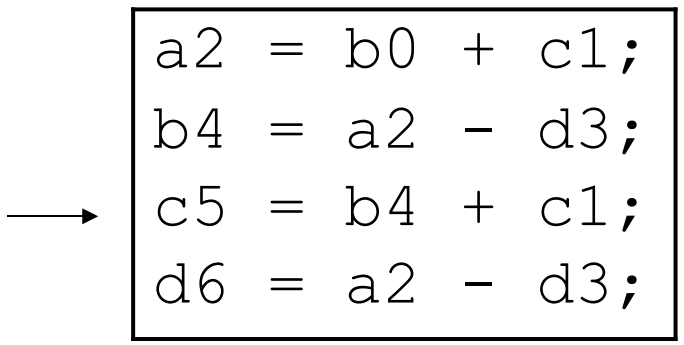
i.e. it is no longer just c = b + c

Local value numbering

Algorithm: Now that variables are numbered

- Iterate sequentially through instructions. Keep a hash table of the rhs (numbered variables and operation) mapped to their lhs.

- At each step, check to see if the rhs has already been computed.




```
H = {
        "b0 + c1" : "a2",
        "a2 - d3" : "b4",
        "b4 + c1" : "c5",
}
```

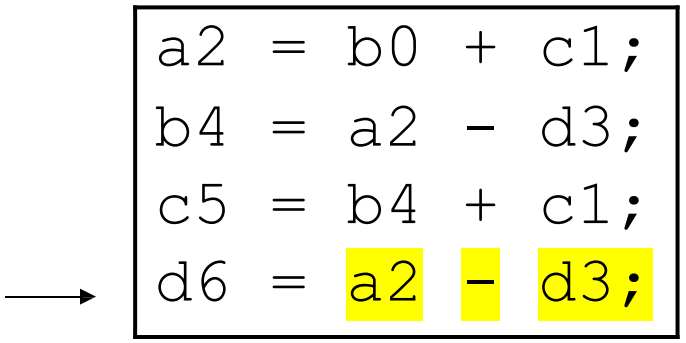
Add new entry in Hash table with new Numbering.

Local value numbering

Algorithm: Now that variables are numbered

- Iterate sequentially through instructions. Keep a hash table of the rhs (numbered variables and operation) mapped to their lhs.

- At each step, check to see if the rhs has already been computed.




```
H = {
        "b0 + c1" : "a2",
        "a2 - d3" : "b4",
        "b4 + c1" : "c5",
}
```

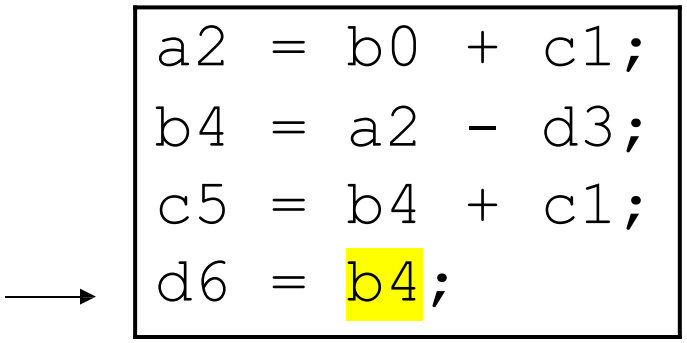
Do a look up and …

Local value numbering

Algorithm: Now that variables are numbered

- Iterate sequentially through instructions. Keep a hash table of the rhs (numbered variables and operation) mapped to their lhs.

- At each step, check to see if the rhs has already been computed.




```
H = {
        "b0 + c1" : "a2",
        "a2 - d3" : "b4",
        "b4 + c1" : "c5",
}
```

…match!

What else can we do?

What else can we do?

Consider this snippet:

```
a2 = c1 - b0;
f4 = d3 * a2;
c5 = b0 - c1;
d6 = a2 * d3;
```

Commutative operations

What is the definition of commutative?

Commutative operations

What is the definition of commutative?

```
x OP y == y OP x
```

What operators are commutative? Which ones are not?

Adding commutativity to local value numbering

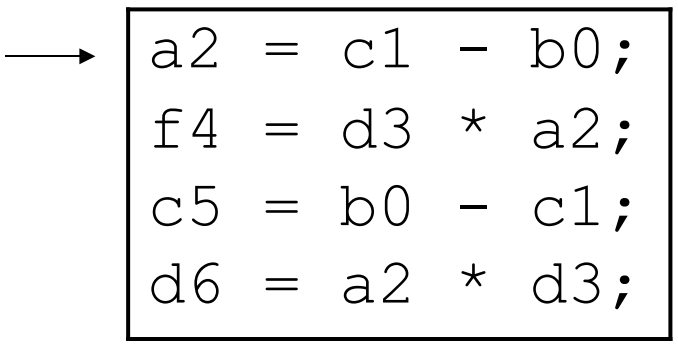
- For commutative operators (e.g. + *), the analysis should consider a deterministic order of operands.

- You can use variable numbers or lexigraphical order

Local value numbering: commutative operations

Algorithm optimization:

- for commutative operations, re-order operands into a deterministic order



```
a2 = c1 - b0;
f4 = d3 * a2;
c5 = b0 - c1;
d6 = a2 * d3;
```

```
H = {
}
```

Local value numbering: commutative operations

Algorithm optimization:

- for commutative operations, re-order operands into a deterministic order

cannot re-order because - is not commutative

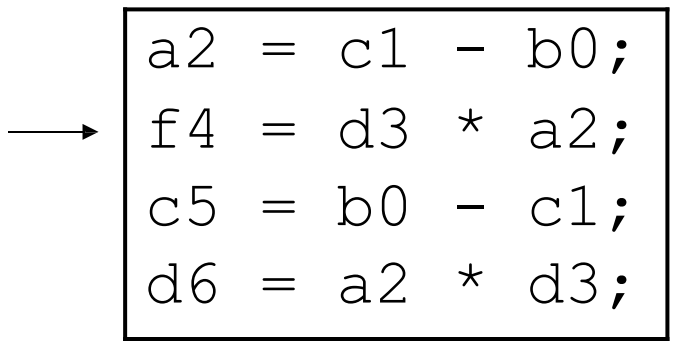
```
a2 = c1 - b0;
f4 = d3 * a2;
c5 = b0 - c1;
d6 = a2 * d3;
```

```
H = {
        “c1 - b0” : “a2”,
}
```

Local value numbering: commutative operations

Algorithm optimization:

- for commutative operations, re-order operands into a deterministic order




```
H = {
        “c1 - b0” : “a2”,
}
```

Local value numbering: commutative operations

Algorithm optimization:

- for commutative operations, re-order operands into a deterministic order

re-ordered because a2 < d3 lexigraphically

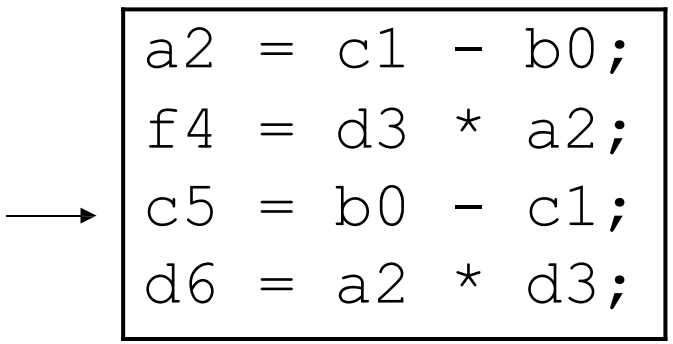
```
a2 = c1 - b0;
f4 = d3 * a2;
c5 = b0 - c1;
d6 = a2 * d3;
```

```
H = {
        "c1 - b0" : "a2",
        "a2 * d3" : "f4",
}
```

Local value numbering: commutative operations

Algorithm optimization:

- for commutative operations, re-order operands into a deterministic order

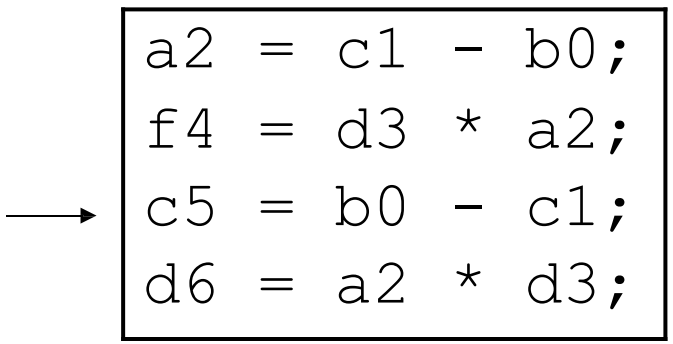



```
H = {
        “c1 - b0” : “a2”,
        ”a2 * d3” : “f4”,
}
```

Local value numbering: commutative operations

Algorithm optimization:

- for commutative operations, re-order operands into a deterministic order

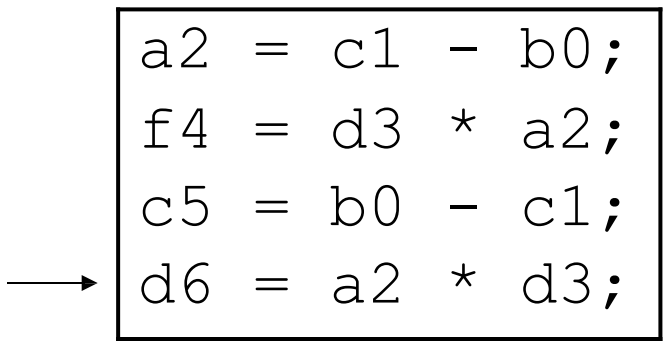



```
H = {
        “c1 - b0” : “a2”,
        ”a2 * d3” : “f4”,
        ”b0 - c1” : “c5”,
}
```

Local value numbering: commutative operations

Algorithm optimization:

- for commutative operations, re-order operands into a deterministic order




```
H = {
        “c1 - b0” : “a2”,
        ”a2 * d3” : “f4”,
        ”b0 - c1” : “c5”,
}
```

Local value numbering: commutative operations

Algorithm optimization:

- for commutative operations, re-order operands into a deterministic order

And achieves more optimization opportunities …

```
a2 = c1 - b0;
f4 = d3 * a2;
c5 = b0 - c1;
d6 = f4;
```

```
H = {
        “c1 - b0” : “a2”,
        ”a2 * d3” : “f4”,
        ”b0 - c1” : “c5”,
}
```

Optimizing over wider regions

- Local value numbering operates over just one basic block.
- We want optimizations that operate over several basic blocks (a region), or across an entire procedure (global)
- For this, we need Control Flow Graphs and Flow Analysis
  - We may have time to discuss this later in the module

Stitching optimized code back into program

How to stitch optimized code back into program

```
a = b + c;
d = e + f;
g = b + c;

label_0:
h = g + a;
k = a + g;
```

How to stitch optimized code back into program

split into basic blocks

```
a = b + c;
d = e + f;
g = b + c;

label_0:
h = g + a;
k = a + g;
```

```
a = b + c;
d = e + f;
g = b + c;
```

```
label_0:
h = g + a;
k = a + g;
```

How to stitch optimized code back into program

Number the names

```
a = b + c;
d = e + f;
g = b + c;

label_0:
h = g + a;
k = a + g;
```

```
a = b + c;
d = e + f;
g = b + c;

label_0:
h = g + a;
k = a + g;
```

```
a2 = b0 + c1;
d5 = e3 + f4;
g6 = b0 + c1;

label_0:
h2 = g0 + a1;
k3 = a1 + g0;
```

How to stitch optimized code back into program

We have moved numbered code
to left on slide to make some room

```
a2 = b0 + c1;
d5 = e3 + f4;
g6 = b0 + c1;
```

```
label_0:
h2 = g0 + a1;
k3 = a1 + g0;
```

How to stitch optimized code back into program

optimized

```
a2 = b0 + c1;
d5 = e3 + f4;
g6 = b0 + c1;
```

```
a2 = b0 + c1;
d5 = e3 + f4;
g6 = a2;
```

```
label_0:
h2 = g0 + a1;
k3 = a1 + g0;
```

```
label_0:
h2 = g0 + a1;
k3 = h2;
```

How to stitch optimized code back into program

```
a2 = b0 + c1;
d5 = e3 + f4;
g6 = b0 + c1;
```

```
label_0:
h2 = g0 + a1;
k3 = a1 + g0;
```

optimized

```
a2 = b0 + c1;
d5 = e3 + f4;
g6 = a2;
```

```
label_0:
h2 = g0 + a1;
k3 = h2;
```

Can it be put together?

```
a2 = b0 + c1;
d5 = e3 + f4;
g6 = a2;

label_0:
h2 = g0 + a1;
k3 = h2;
```

What are the resulting issues?

How to stitch optimized code back into program

```
a2 = b0 + c1;
d5 = e3 + f4;
g6 = b0 + c1;
```

```
label_0:
h2 = g0 + a1;
k3 = a1 + g0;
```

optimized

```
a2 = b0 + c1;
d5 = e3 + f4;
g6 = a2;
```

```
label_0:
h2 = g0 + a1;
k3 = h2;
```

Can it be put together?

```
a2 = b0 + c1;
d5 = e3 + f4;
g6 = a2;

label_0:
h2 = g0 + a1;
k3 = h2;
```

What are the resulting issues? undefined!

How to stitch optimized code back into program

Let's fix it

stitch part 1: ***assign original unnumbered variables their latest values in each basic block***

```
a2 = b0 + c1;
d5 = e3 + f4;
g6 = b0 + c1;
```

```
label_0:
h2 = g0 + a1;
k3 = a1 + g0;
```

```
a2 = b0 + c1;
d5 = e3 + f4;
g6 = a2;
```

```
label_0:
h2 = g0 + a1;
k3 = h2;
```

```
a2 = b0 + c1;
d5 = e3 + f4;
g6 = a2;
g = g6;
d = d5
a = a2;
```

```
label_0:
h2 = g0 + a1;
k3 = h2;
h = h2;
k = k3;
```

How to stitch optimized code back into program

We have moved modified basic blocks to left on slide to make some room

```
a2 = b0 + c1;
d5 = e3 + f4;
g6 = a2;
g = g6;
d = d5
a = a2;
```

```
label_0:
h2 = g0 + a1;
k3 = h2;
h = h2;
k = k3;
```

But what else needs to be done?

We have a problem …

How to stitch optimized code back into program

We have moved modified basic blocks to left on slide to make some room

```
a2 = b0 + c1;
d5 = e3 + f4;
g6 = a2;
g = g6;
d = d5
a = a2;
```

```
label_0:
h2 = g0 + a1;
k3 = h2;
h = h2;
k = k3;
```

We need to get the input variables used by the basic blocks, to have their unnumbered values: 2 ways to do it …

How to stitch optimized code back into program

Method 1:

```
a2 = b0 + c1;
d5 = e3 + f4;
g6 = a2;
g = g6;
d = d5
a = a2;
```

```
label_0:
h2 = g0 + a1;
k3 = h2;
h = h2;
k = k3;
```

```
b0 = b
c1 = c
e3 = e
f4 = f
a2 = b0 + c1;
d5 = e3 + f4;
g6 = a2;
g = g6;
d = d5
a = a2;
```

Continue onto label_0
of other basic block

stitch part 2:

Copy unnumbered inputs to initial RHS numbered inputs

```
label_0:
g0 = g
a1 = a
h2 = g0 + a1;
k3 = h2;
h = h2;
k = k3;
```

How to stitch optimized code back into program

Method 2: stitch part 2: **drop numbers from first use of RHS variables**

```
a2 = b0 + c1;
d5 = e3 + f4;
g6 = a2;
g = g6;
d = d5
a = a2;
```

```
a2 = b + c;
d5 = e + f;
g6 = a2;
g = g6;
d = d5
a = a2;
```

```
label_0:
h2 = g0 + a1;
k3 = h2;
h = h2;
k = k3;
```

```
label_0:
h2 = g + a;
k3 = h2;
h = h2;
k = k3;
```

How to stitch optimized code back into program

Now the Basic Blocks can be re-combined!!

```
a2 = b0 + c1;
d5 = e3 + f4;
g6 = a2;
g = g6;
d = d5
a = a2;
```

```
a2 = b + c;
d5 = e + f;
g6 = a2;
g = g6;
d = d5
a = a2;
```

```
label_0:
h2 = g0 + a1;
k3 = h2;
h = h2;
k = k3;
```

```
label_0:
h2 = g + a;
k3 = h2;
h = h2;
k = k3;
```

```
a2 = b + c;
d5 = e + f;
g6 = a2;
g = g6;
d = d5
a = a2;

label_0:
h2 = g + a;
k3 = h2;
h = h2;
k = k3;
```

How to stitch optimized code back into program

original

```
a = b + c;
d = e + f;
g = b + c;

label_0:
h = g + a;
k = a + g;
```

new

```
a2 = b + c;
d5 = e + f;
g6 = a2;
g = g6;
d = d5;
a = a2;
label_0:
h2 = g + a;
k3 = h2;
h = h2;
k = k3;
```

But is it really optimized?

It looks a lot longer...

How to stitch optimized code back into program

original

```
a = b + c;
d = e + f;
g = b + c;

label_0:
h = g + a;
k = a + g;
```

new

```
a2 = b + c;
d5 = e + f;
g6 = a2;
g = g6;
d = d5
a = a2;
label_0:
h2 = g + a;
k3 = h2;
h = h2;
k = k3;
```

But is it really optimized?

Common pattern is for code to get larger, but it will contain patterns that are easier to optimize away

later passes will minimize copies

e.g. copy propagation future pass eliminates repeats of variables.